

**Energy-Efficient Dual Hydroponic System for
Organic/Inorganic Capsicum Chili Farming**

25-26J-035

Project Proposal Report

Galappaththi D.S

BSc (Hons) in Information Technology

Specializing in Computer Systems & Network Engineering

Department of Computer Systems Engineering

Sri Lanka Institute of Information Technology

Sri Lanka

August 2025

**Energy-Efficient Dual Hydroponic System for
Organic/Inorganic Capsicum Chili Farming**

25-26J-035

Project Proposal Report

Galappaththi D.S – IT22355782

Supervisor: Ms. Shashika Lokuliyana

Co-Supervisor: Mr. Dinith Primal

BSc (Hons) in Information Technology

Specializing in Computer Systems & Network Engineering

Department of Computer Systems Engineering

Sri Lanka Institute of Information Technology

Sri Lanka

August 2025

DECLARATION

I declare that this is my own work, and this proposal does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Name	Student ID	Signature
Galappaththi D.S	IT22355782	<i>Dela</i>



Signature of the Supervisor

19/9/25

Date

ABSTRACT

Hydroponic farming benefits greatly from IoT monitoring, but off-grid deployments face two stubborn constraints: limited solar energy and unreliable connectivity. Traditional, connection-oriented transports and single-link designs burn power on handshakes and keepalives and create a single point of failure. At the same time, unsynchronized sensing across multiple tanks makes side-by-side comparisons noisy or misleading. In real farm settings, the communication layer must therefore be both low-power and resilient, while keeping measurements aligned in time.

This project designs a solar-powered ESP32 platform built around a UDP-based lightweight application-layer protocol that minimizes packet and handshake overhead. An energy-aware scheduler adapts sensing and sending rates to battery state and harvest conditions, maximizing deep sleep when energy is scarce. A firmware-level quality-of-service policy elevates critical events (pump failure, low water level, extreme pH/EC) so they preempt queues, request acknowledgments, and can trigger failover when needed. Connectivity resilience is provided by dual-link operation—Wi-Fi as the primary path and GSM as backup—managed by a simple state machine that activates the secondary link only during outages. Opportunistic network time synchronization keeps device clocks aligned so readings from organic and inorganic tanks can be compared fairly.

The system is expected to demonstrate a 20–40% reduction in communication-related energy use and markedly higher delivery rates for critical alerts compared with fixed-frequency, single-link designs. The result is a practical, low-cost communication backbone that improves reliability, data integrity, and sustainability for dual hydroponic systems. Because the approach is modular—lightweight transport, energy-aware scheduling, device-side QoS, dual-link failover, and time sync—it can be adapted to other smart-agriculture and remote sensing scenarios where power and connectivity are limited.

Keywords: IoT Hydroponics; UDP-based Lightweight Protocol; Energy-Aware Scheduling; Dual-Link Failover (Wi-Fi/GSM); Time Synchronization

TABLE OF CONTENT

DECLARATION.....	iii
ABSTRACT.....	iv
TABLE OF CONTENT	v
LIST OF FIGURES.....	vii
LIST OF TABLES.....	viii
LIST OF ABBREVIATIONS	ix
1. INTRODUCTION.....	1
1.1 Background	2
1.2 Literature Survey.....	4
1.3 Research Gap.....	7
1.4 Research Problem.....	9
1.5 Research Questions	10
2. OBJECTIVES	11
2.1 Main Objective.....	11
2.2 Sub-Objectives	11
3. METHODOLOGY.....	12
3.1 System Diagram	12
3.2 Overall System Description	13
3.3 Component Development and Tasks.....	13
3.4 Data Collection and Evaluation Plan	15
3.5 Work Breakdown.....	16
3.6 Time Frame	19
.....	19
4. Project requirements.....	20
4.1 Functional Parameters (how the system behaves).....	20
4.2 Non-Functional Parameters (how well it should work)	21
4.3 User Requirements	22

4.4	System Requirements	22
4.5	Use Cases	23
4.6	Test Cases (Tentative)	24
4.7	Wireframes (Tentative).....	25
5.	DESCRIPTION OF PERSONNEL AND FACILITIES	26
5.1	Personnel	26
5.2	Facilities	27
6.	BUDGET AND BUDGET JUSTIFICATION	28
6.1	Estimated Project Budget	28
6.2	Budget Justification.....	29
7.	CONCLUSIONS AND RECOMMENDATIONS	30
7.1	Conclusions	30
7.2	Recommendations	31
	REFERENCES.....	32

LIST OF FIGURES

Figure 3.1: System Diagram.....	12
Figure 3.2: Gantt Chart.....	19

LIST OF TABLES

Table 6.1: Estimated Project Budget	29
---	----

LIST OF ABBREVIATIONS

Abbreviation	Description
ADC	Analog-to-Digital Converter
API	Application Programming Interface
CRC	Cyclic Redundancy Check
DHT	Digital Humidity & Temperature sensor
EC	Electrical Conductivity
ESP32	Espressif Systems 32-bit Microcontroller
GSM	Global System for Mobile Communication
GUI	Graphical User Interface
HTTP	Hypertext Transfer Protocol
IoT	Internet of Things
LPWAN	Low Power Wide Area Network
MQTT	Message Queuing Telemetry Transport
NTP	Network Time Protocol
pH	Potential of Hydrogen (acidity/alkalinity measure)
PV	Photovoltaic (solar panel technology)
QoS	Quality of Service
RTC	Real-Time Clock
SMS	Short Message Service
SoC	System on Chip
TCP	Transmission Control Protocol
UDP	User Datagram Protocol
UI	User Interface
WLAN	Wireless Local Area Network
WSN	Wireless Sensor Network

1. INTRODUCTION

The use of Internet of Things (IoT) in agriculture is rapidly growing due to its potential to improve productivity, optimize resources, and reduce manual monitoring. However, in developing regions such as Sri Lanka, challenges such as energy scarcity and network unreliability make traditional IoT approaches difficult to sustain. Conventional protocols like TCP/IP consume excessive energy due to connection overhead, while reliance on a single communication channel, such as Wi-Fi, introduces a single point of failure that threatens system reliability [1], [5], [9].

This project addresses these limitations by developing a lightweight UDP-based communication framework integrated with adaptive power management and a dual-failover mechanism (Wi-Fi + GSM). Prior studies have highlighted the importance of adaptive duty cycling and energy-aware scheduling in prolonging IoT device lifetime, especially in solar-powered or energy-harvesting systems [2], [4], [7], [8]. A Quality of Service (QoS) layer ensures that critical sensor alerts are always prioritized, while less urgent data is transmitted adaptively depending on energy availability [13], [14]. The system also supports time synchronization for accurate comparative monitoring of dual hydroponic tanks (organic and inorganic), enabling reliable data-driven insights [3], [12], [17].

By combining energy efficiency, resilience, and intelligent data handling, this work lays the communication foundation for a sustainable, solar-powered hydroponic monitoring system that can operate effectively in off-grid agricultural environments [10], [11], [15], [18].

1.1 Background

The Internet of Things (IoT) has reshaped multiple domains by enabling widespread sensing and control. In agriculture, IoT-driven hydroponics uses continuous measurements of pH, EC, temperature, water level, and flow to deliver resource-efficient cultivation with improved yield stability. These benefits are compelling in real farms and greenhouses especially in off-grid areas in Sri Lanka yet deployments face two persistent constraints: tight energy budgets and unreliable connectivity. On MCU-class nodes like the ESP32, energy is dominated by radio activity; associations, handshakes, retransmissions, and idle listening often dwarf the cost of the sensor reading. Research on energy-efficient IoT shows that protocol choices, sleep policy, and workload scheduling determine whether a node remains energy-neutral under solar harvesting or gradually depletes its battery [1]. Energy-aware frameworks and measured results from harvested or battery-less networks demonstrate that adapting the duty cycle to available energy significantly improves stability and lifetime compared to fixed-rate operation [2]. Formal scheduling (e.g., MDP-based) optimizes sampling and transmission against battery dynamics [4], and broader smart-energy studies reinforce the same principle: schedule the node's work to the harvester and the state-of-charge rather than to a rigid clock [1], [15].

Communication stack selection compounds the energy story. Connection-oriented transports such as TCP require handshakes, keepalives, and per-message state, raising both wireless-active time and bytes per useful payload. Agricultural communication studies comparing protocol families report that lower-overhead, UDP-centric designs reduce per-message cost for small, occasional sensor data uploads on constrained devices [10], while recognizing that naive fire-and-forget UDP is unsafe in lossy fields. Hardware evaluations of enhanced UDP show that adding minimal acknowledgment logic (selective ACK/NAK, bounded retries) improves delivery at a modest compute/energy cost—far below running full TCP flows for every message [5], [9]. This favors a UDP-based lightweight application-layer protocol that trims handshakes and headers for routine sensor data but selectively confirms critical events so they are not lost.

Resilience is equally important. Wi-Fi coverage on farms is often unreliable, AP resets, and upstream outages; a single access path becomes a single point of failure. Redundancy/failover literature shows that monitoring per-link service levels and switching interfaces (e.g., Wi-Fi→GSM) preserves continuity for critical traffic during failures, often combined with payload adaptation so only high-priority alerts traverse the backup link [6], [13], [18]. For off-grid hydroponics, a practical policy is to keep GSM largely idle to save power and data, waking it only when a priority alert must be delivered or when repeated Wi-Fi ACK failures indicate the primary is down. This event-driven failover controls overhead while maintaining safety-critical visibility.

Placing QoS in the firmware (device-side) gives the ESP32 authority to prioritize before packets hit the network: high-priority events (pump failure, water shortage, extreme pH/EC) preempt queues, request confirmations, and can trigger failover when primary ACKs are missing, while low-priority sensor data is batched, deferred, or dropped under energy stress. Coupled with a voltage-tiered scheduler (High/Medium/Low SoC states), the node modulates sampling and transmission and maximizes deep sleep according to harvest and battery status—keeping radio-on time low and still guaranteeing alert delivery [1], [2], [4], [15]. This satisfies the energy constraint without sacrificing reliability.

Finally, the project’s goal of comparing two NFT tanks (organic vs. inorganic) requires timestamp alignment. Without synchronized clocks, environmental variations can be misattributed to treatment effects. NTP/SNTP on lossy wireless links can struggle because of variable RTTs; recent work surveys IoT-appropriate clock-discipline and improved NTP strategies for changing network conditions [3], [12], [17]. On ESP32, opportunistic SNTP—synchronizing during good link and energy states—keeps drift bounded at low cost. Embedding timestamps in each payload and aligning server-side preserves the scientific validity of cross-tank comparisons [3], [12], [17].

In summary, the literature across energy-efficient IoT and agriculture converges on four practical choices for off-grid hydroponics: (1) reduce protocol overhead with a UDP-based lightweight application-layer protocol using selective acknowledgments only for priority data [1], [5], [9], [10]; (2) schedule to the energy envelope with

voltage-tiered duty cycling and deep sleep [1], [2], [4], [15]; (3) engineer resilience via dual-link failover (Wi-Fi primary, GSM backup) and device-side QoS so alerts always have a delivery path [6], [13], [18]; and (4) keep clocks aligned through opportunistic network time sync for valid dual-tank analysis [3], [12], [17]. These choices are low-cost and feasible on ESP32 + SIM800L and create measurable baselines (radio-active minutes/day, mWh/day, packet delivery/latency, clock error) against fixed-frequency, single-link, TCP-based designs [1]–[4], [6]–[10], [15], [17], [18].

1.2 Literature Survey

Communication overhead vs. energy on constrained nodes

Protocol costs. Kaur & Swamy [1] survey low-power IoT stacks and argue that connection-oriented transports (e.g., TCP) add handshakes/keepalives that inflate energy on MCU-class devices. Their synthesis recommends lightweight, UDP-centric telemetry and sleep-first design. Yuksel [9] measures a Wi-Fi IoT device and shows that radio activity dominates energy, reinforcing that per-message connection setup is costly.

UDP with minimal reliability. Adetunji et al. [5] implement UDP with acknowledgments across multiple topologies and report that selective ACK/NAK can improve delivery without the heavy cost of TCP. The improvement is topology-dependent, but the key takeaway is that bounded, application-layer reliability over UDP strikes a better energy/reliability balance on tiny nodes than always using TCP.

Limitation/loophole: [5] focuses on network performance; detailed battery-level energy deltas are not their main metric.

Implication for our method: Use a UDP-based lightweight application-layer protocol with selective ACK/NAK only for priority traffic, avoiding TCP’s systematic overhead while preserving reliability for alerts ([1], [5], [9], [10]). Agriculture-specific protocol trade-offs. The outdoor agriculture comparison in [10] contrasts Wi-Fi, ZigBee/LoRa, and other stacks in field conditions, showing Wi-Fi’s accuracy and integration strengths versus LoRaWAN’s long-range/low-energy benefits. Singh et al. [18]

validate LoRaWAN in greenhouses for low-rate telemetry, highlighting its energy efficiency but limited bandwidth for richer data.

Limitation: LoRaWAN’s low throughput limits multi-sensor or time-dense hydroponic analytics.

Implication: Keep Wi-Fi as primary for richer uploads, but design low-overhead packets and deep sleep to make Wi-Fi viable; use cellular failover only for alerts ([10], [18]).

Energy-aware scheduling and power management

Adaptive duty-cycling. Van Leemput et al. [2] (battery-less 6TiSCH) demonstrate energy-aware scheduling that adapts to energy availability, improving stability vs. fixed periodicity. Charoenchaiprakit et al. [4] use an MDP policy that optimizes duty cycle vs. battery energy, showing better continuity and service compared to naive schedules. Broader surveys in [1] and solar/harvesting-aware forecasting in [15] advocate matching sampling/transmission to harvest and state-of-charge rather than a rigid clock.

Limitation: [2] targets 6TiSCH/industrial stacks; [4] relies on MDPs that may be heavyweight on small MCUs without simplification.

Implication: Implement a Voltage-Tiered Scheduler (VTS) on ESP32—High/Medium/Low SoC states that modulate sensing/telemetry and deep sleep—capturing the spirit of [2]/[4] with a lightweight policy practical on MCU firmware ([1], [2], [4], [15]). Task scheduling and harvesting. The survey in [8] reviews co-scheduling sensing with energy harvesting, emphasizing that harvest signals should inform scheduling decisions.

Limitation: Framework-level guidance; less about concrete MCU code paths.

Implication: Our VTS uses battery/harvest status as direct inputs (voltage/current), keeping the implementation simple while following the survey’s principle ([8]).

Resilience, redundancy, and device-side QoS

Failover across radios. Ember/Boano et al. [6] present communication failover strategies (smart grid context) showing that multi-radio redundancy and service-level monitoring preserve critical traffic during failures. Rullo et al. [13] review redundancy for IoT fault-tolerance, covering multi-pathing and replication to avoid single points of failure.

Limitation: Generic frameworks—often not tailored to battery-constrained MCUs.

Implication: We adopt a dual-link failover pattern (Wi-Fi → GSM) with event-driven activation: keep GSM off to save power, wake it only for priority alerts or prolonged Wi-Fi ACK failure. This embodies resilience from [6]/[13] in a power-aware form suitable for ESP32.

QoS under energy constraints. Rai & Sarma [14] survey QoS prediction/management for WSNs, balancing latency, reliability, and energy. Their insight: not all data is equal—protocols should prioritize.

Limitation: Emphasis on modeling/prediction; limited device-firmware recipes.

Implication: Implement device-side QoS in firmware: priority tagging, preemptive sending for alerts, selective ACK/NAK, rate shaping and buffering for non-critical data—so scarce energy serves the most important information first ([14], supported by [5]).

Time synchronization and data integrity for comparative analytics

NTP under variable RTT. The dynamic NTP framework in *Sensors* [3] and the IIoT-tuned NTP in *Sensors* [12] address clock discipline under fluctuating latencies, improving time accuracy in lossy wireless environments. Yiğitler et al. [17] provide a broad overview of clock discipline algorithms and protocols for IoT deployments. Limitation: Some approaches assume more stable links or server assist than farms routinely have.

Implication: Use opportunistic SNTP on ESP32—sync in good link/energy windows—and embed timestamps in every UDP packet; this keeps energy low while maintaining cross-tank comparability ([3], [12], [17]).

Agriculture context. Field-oriented studies emphasize that valid agronomic comparisons require aligned data streams; in practice, timestamp discipline is as important as sensor accuracy ([10], [18]).

Implication: Our pipeline aligns with this: timestamp at source, server-side alignment, and periodic re-discipline.

Hydroponics + renewable energy experience

Solar-powered hydroponics prototypes. Melkysedek et al. [11] built a solar-powered hydroponic installation (NodeMCU + sensors), confirming feasibility but lacking optimized communication or link failover—so gaps/overheads remain. Sishodia et al. [16] show IoT-based hydroponics improves productivity and monitoring but do not address energy-aware scheduling or priority-driven delivery. Gholami et al. / Shirbhate & Barve [15] present solar monitoring and energy prediction, helpful for forecasting but often compute-heavier than what an ESP32 can run continuously. Implication: These works validate the domain need (hydroponics + solar), but leave the communication-energy-resilience triad open—precisely what our design targets with UDP-based lightweight transport, VTS scheduling, device-side QoS, and dual-link failover.

1.3 Research Gap

The current body of research and commercial solutions for IoT-based smart agriculture shows several gaps that limit practical, off-grid hydroponic deployments. Our work addresses the following:

Missing integration of energy awareness with communication policy

Most systems treat sensing/automation and power management separately. They use fixed sampling/tx periods or only basic deep sleep, failing to adapt to changing solar input and state-of-charge. Literature calls for adaptive duty-cycling and forecasting, but lightweight MCU-level implementations that directly couple energy context to communication behavior remain scarce [1], [2], [4], [15].

- Our contribution: a Voltage-Tiered Scheduler (VTS) that modulates sensing/tx and deep sleep by SoC/harvest state.
- Target metrics: $\geq 20\%$ reduction in radio-active minutes/day and mWh/day versus a fixed-rate baseline.

Inefficient transports for off-grid devices

Connection-oriented transports such as TCP add handshakes/keepalives that raise radio-on time and drain batteries on MCU nodes. UDP is lighter, but most studies stop at general comparisons; few present domain-specific, embedded reliability tuned for agriculture [5], [9], [10].

- Our contribution: a UDP-based lightweight application-layer protocol with selective ACK/NAK only for priority packets and bounded retries.
- Target metrics: 20–40% reduction in communication energy vs. TCP baseline; equal or higher delivery rate for alerts.

Single-link designs and weak resilience

Low-cost deployments typically rely on a single Wi-Fi link, creating a single point of failure. Failover has been studied in smart grid/industrial contexts, but not as a power-aware, event-driven pattern on low-cost hydroponic nodes [6], [13].

- Our contribution: dual-link failover (Wi-Fi→GSM) that activates GSM only for critical alerts or sustained Wi-Fi ACK loss.
- Target metrics: $\geq 99\%$ alert delivery during induced Wi-Fi outages with $< 5\%$ increase in daily energy vs. no-failover design.

Lack of lightweight, device-side QoS

Many systems “send everything” regardless of urgency or battery state, wasting energy and risking missed alerts during low-power periods. Existing QoS work often assumes heavier stacks or ignores energy constraints on MCUs [10], [14].

- Our contribution: firmware-level (device-side) QoS: priority tagging, preemptive send for alerts, selective confirmation, batching/defer for routine data under low SoC.
- Target metrics: 100% delivery of critical events under Low-power state; $\geq 50\%$ reduction in low-priority bytes during Low state.

Time synchronization for dual-tank comparisons

Comparative hydroponics (organic vs. inorganic tanks) requires timestamp-aligned streams. Conventional NTP can degrade under variable RTTs on lossy Wi-Fi; IIoT-tuned methods exist, but adoption on ESP32-class nodes with opportunistic policies is limited [3], [12], [17].

- Our contribution: opportunistic SNTP (sync only in good link/energy windows) and timestamp-at-source for every packet.
- Target metrics: $\leq \pm 200$ ms inter-tank skew during 24-hour runs; stable alignment despite induced link variability.

1.4 Research Problem

IoT-based hydroponic monitoring systems deployed in off-grid environments suffer from high energy consumption, unreliable communication, and poor data resilience. Existing approaches often rely on TCP or static duty cycling, which are inefficient for solar-powered nodes—causing rapid battery depletion and communication breakdowns [1], [4], [9]. Moreover, most low-cost systems use a single communication path (e.g., Wi-Fi only), creating a single point of failure during connectivity outages [6], [13]. Finally, unsynchronized timestamps across tanks degrade the validity of comparative analytics under variable wireless latency [3], [12]. Together, these limitations prevent reliable delivery of critical alerts, undermine system sustainability, and hinder the scalability of smart-agriculture solutions.

Therefore, the central research problem is the absence of a lightweight, adaptive, and resilient communication framework that:

- minimizes packet and handshake overhead using a UDP-based lightweight application-layer protocol,
- dynamically adapts sensing and data transmission to energy availability (harvest/SoC),
- ensures reliability under outages via multi-channel failover (Wi-Fi primary, GSM backup) with device-side QoS so critical data is always delivered, and
- maintains timestamp alignment through opportunistic network time synchronization for valid cross-tank comparisons.

This project addresses the problem by developing a low-power, energy-aware networking module for dual hydroponic tanks that integrates UDP-based lightweight transport, voltage-tiered scheduling, device-side QoS, dual-link failover, and opportunistic time sync—enabling sustainable and resilient IoT operations in resource-constrained agricultural environments.

1.5 Research Questions

1. RQ1 (Transport efficiency): Does the UDP-based lightweight protocol reduce per-message bytes and radio-on time on the ESP32 compared with TCP for the same payloads?
2. RQ2 (Energy-aware operation): Does the Voltage-Tiered Scheduler (VTS-3) maintain continuous solar operation and lower daily energy use compared with a fixed-rate schedule?
3. RQ3 (Resilient alerts): During Wi-Fi outages, does GSM failover deliver $\geq 99\%$ of critical alerts within acceptable latency?
4. RQ4 (Time alignment): Over 24 hours, does the system keep inter-tank timestamp skew within ± 200 ms using opportunistic SNTP?

2. OBJECTIVES

2.1 Main Objective

To design and implement a low-power, resilient communication and power-management module for a solar-powered, dual-tank hydroponic system that sustains reliable monitoring and timely alerts in off-grid conditions..

2.2 Sub-Objectives

- **Transport efficiency**
To develop a UDP-based lightweight application-layer protocol with selective ACK/NAK and compact headers that reduces per-message energy and handshake overhead compared with TCP for routine sensor data and alerts.
- **Energy-aware operation**
To implement a voltage-tiered scheduler (VTS) that adapts sampling, transmission, and deep-sleep intervals to battery/harvest state (High/Medium/Low) to maintain energy neutrality.
- **Reliability & resilience**
To ensure dependable delivery of critical events by integrating device-side QoS with dual-link failover (Wi-Fi primary, GSM backup), activating the backup path only for priority alerts or sustained primary-link failures.
- **Time alignment for comparisons**
To maintain cross-tank data validity by implementing opportunistic network time synchronization (SNTP) and timestamp-at-source, enabling accurate, synchronized analytics between organic and inorganic tanks.

3. METHODOLOGY

3.1 System Diagram

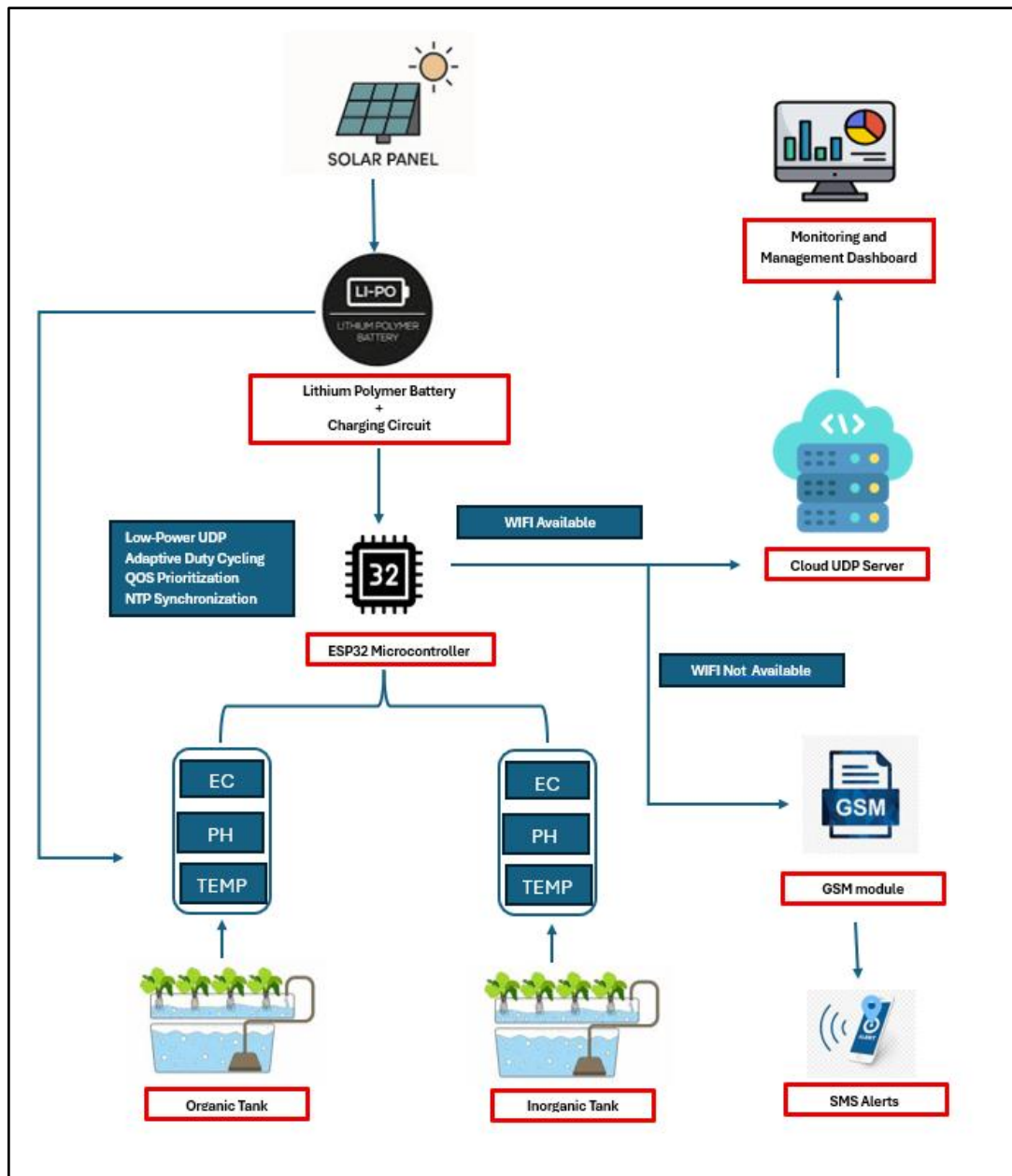


Figure 3.1: System Diagram

3.2 Overall System Description

Figure 3.1 shows a solar panel , Li-Po battery + charger , ESP32 that serves two NFT tanks (organic and inorganic). Each tank has sensors for pH, EC, temperature, and water level. The ESP32 communicates primarily via Wi-Fi to a Cloud UDP server and uses a GSM/SIM800L backup when Wi-Fi is unavailable. The firmware runs a UDP-based lightweight application-layer protocol, a Voltage-Tiered Scheduler (VTS-3), device-side QoS, and opportunistic SNTP time sync. The cloud stack ingests packets and powers a web dashboard for monitoring and analysis.

3.3 Component Development and Tasks

Sensing and Data Acquisition.

Each tank uses calibrated pH and EC analog front-ends, a DS18B20 temperature probe, and a float-switch water-level input. The ESP32 polls sensors on wake, applies calibration/linearization, and encodes results as scaled integers. Readings are bundled into compact UDP payloads with device ID, sequence number, and timestamp to minimize bytes on air [11], [16], [18].

Low-Power UDP Communication Protocol.

The node implements a UDP-based lightweight application-layer protocol with a small header (version, type, device ID, seq, timestamp, flags, length, CRC) and three message types: ALERT, STATE (battery/link), and DATA (sensor batch). Reliability is selective: only ALERT requires ACK/NAK with bounded retries; STATE/DATA are best-effort and may be batched. This avoids TCP handshakes/keepalives that raise radio-active time while maintaining dependable alert delivery—matching evidence from energy-aware IoT and UDP+ACK studies [1], [5], [9], [10].

Adaptive Power Management.

Because power is harvested, operation follows a Voltage-Tiered Scheduler (VTS): the ESP32 samples battery voltage and adapts sampling/transmission and deep sleep accordingly. In a typical profile—High SoC: full sensing, transmit ~5 min; Medium: down-sample non-critical streams and batch tx (e.g., 30 min); Low: send ALERT only, buffer routine data, maximize sleep. This implements the literature’s conclusion that

adaptive duty-cycling outperforms fixed-rate operation in harvested systems [2], [4], [15].

Resilient Dual-Failover Connectivity.

Wi-Fi is the primary path to the UDP server. If three consecutive ALERT packets miss Wi-Fi ACKs (or periodic pings fail), the node activates GSM/SIM800L to relay the alert via GSM data or SMS, then reverts to Wi-Fi after several successful pings. This event-driven failover removes the single point of failure yet keeps the backup radio off most of the time—consistent with resilience guidance in dependability work [6], [13], [18].

QoS and Data Prioritization.

A firmware-level (device-side) QoS policy tags every record with a priority. High-priority events (pump failure, low water level, extreme pH/EC) preempt queues, demand ACK/NAK, and can trigger GSM failover when Wi-Fi is impaired; lower-priority streams are batched, deferred, or dropped in Low-power state. This aligns scarce energy with information value and reflects QoS findings for constrained IoT/WSN [14], together with the selective-ACK approach above [5].

Synchronization Between Tanks.

Each node runs opportunistic SNTP: synchronize only in High/Medium energy and good link windows; timestamp at source for every packet; align streams server-side. This follows improved NTP/clock-discipline guidance for variable-latency IoT links and maintains valid cross-tank comparisons [3], [12], [17].

Cloud Data Collection and Dashboard.

A Python FastAPI UDP ingestor (uvicorn) validates CRC and writes to PostgreSQL. A React dashboard renders real-time charts for pH/EC/temperature/level, power status, link state, retries/ACKs, and an alerts panel. This stack keeps the path simple and auditable for experiments [10], [18].

The Algorithm: Voltage-Tiered Scheduler (VTS)

Goal. Keep the node energy-neutral while guaranteeing alert delivery under changing harvest/SoC [2], [4], [15].

Inputs. Battery voltage V_b (ADC), optional charge current I_{bI} , recent link health (ACK success), and QoS priority.

States & Thresholds (tunable).

- High: $V_b \geq 4.05 \text{ V}$ $\rightarrow$ dense sensing; every 5 min; SNTP allowed.
- Medium: $3.80 \leq V_b < 4.05 \text{ V}$ $\rightarrow$ down-sample non-critical; batch 30 min; limited SNTP.
- Low: $V_b < 3.80 \text{ V}$ $\rightarrow$ ALERT-only; buffer routine data; deep-sleep max; GSM failover permitted.

3.4 Data Collection and Evaluation Plan

Records

For both tanks, the system logs: pH, EC, water level, temperature, battery level, sending rate (how often data is sent), active link (Wi-Fi or GSM), bytes sent, retries/ACKs, messages delivered vs. lost, and end-to-end latency (time from sensor read to dashboard). The server saves every packet with a receive timestamp so results can be verified and plotted.

Metrics

- Energy use: total energy per day (mWh/day), radio-on minutes per day, battery-voltage trace, and time spent in each VTS-3 state (High/Medium/Low).
- Reliability: delivery rate (delivered $\div$ sent) and alert latency (sensor $\rightarrow$ dashboard) for Wi-Fi and, when used, GSM.

- Time alignment: timestamp difference between the two tanks over a 24-hour window.
- Data integrity: CRC error rate, duplicates, and sequence gaps.

Pass / fail targets

- Uptime & energy: no brownouts during the pilot; continuous 24/7 solar operation for ≥ 72 hours.
- Reliability: $\geq 99\%$ delivery of critical alerts; median alert latency ≤ 5 s (Wi-Fi) and ≤ 15 s (GSM).
- Time sync: inter-tank timestamp skew $\leq \pm 200$ ms over 24 hours.

All raw logs (node timestamps and server receive times) will be archived to support verification and visualization of results.

3.5 Work Breakdown

Phase 1 — Plan the system (July - September 2025)

Goal: Agree on what we're building and how it fits together.

- Finalize the system diagram and name each connection.
- Write the message format for UDP (what fields go in a packet).
- Decide the battery levels for VTS-3 (High / Medium / Low).
- List the measures we will collect (energy, delivery, latency, time skew).

Lead: Networking lead

Output: Architecture diagram, protocol sheet, test/metrics plan, parts list

Phase 2 — Build the hardware (September – November 2025)

Goal: Make the physical system work on a bench.

- Wire solar ,charger ,battery , ESP32 and check power.
- Connect and calibrate sensors (pH, EC, temperature, water level).
- Put everything in a safe, weather-resistant enclosure.

Lead: Hardware lead

Output: Powered, calibrated hardware for each tank

Phase 3 — Write the device software (September - December 2025)

Goal: Make the ESP32 behave correctly in the field.

- Implement the UDP messages with a checksum.
- Add priority & acknowledgments for alerts (QoS + selective ACK).
- Add the VTS-3 scheduler (behave differently at High/Med/Low battery).
- Add Wi-Fi primary and GSM backup (automatic failover).
- Add time sync (SNTP) and timestamp every packet.

Lead: Networking lead

Output: Firmware that can send data, save power, fail over, and keep time

Phase 4 — Set up the cloud and dashboard (November – February 2026)

Goal: Receive data and show it clearly.

- Run a UDP Receiver (FastAPI) that checks packets and stores them.
- Create the database tables (sensor values, battery, link, alerts).
- Build the dashboard (live tiles, charts, alert panel, history + CSV).
- Turn on HTTPS and passwords.

Lead: Backend/Frontend lead

Output: Working server + database + web dashboard

Phase 5 — Join it all up & test indoors (November - December 2025)

Goal: Prove end-to-end flow before going outside.

- Connect device → Wi-Fi ,UDP Receiver ,database, dashboard.
- Simulate Wi-Fi outages, low battery, and alerts.

Lead: Test lead

Output: Bench test report with logs and screenshots

Phase 6 — Run a short field pilot (January - March 2026)

Goal: See it work on the two real tanks.

- Install both nodes on the organic and inorganic tanks.
- Run for at least 72 hours (day/night, sun/cloud).
- Watch the dashboard and fix minor issues.

Lead: Hardware + Networking

Output: Real dataset from both tanks; stable behavior

Phase 7 — Evaluate and write up (March - May 2026)

Goal: Show results clearly.

- Calculate daily energy, radio-on minutes, delivery %, alert delay, time skew.
- Make plots/tables and write the results, discussion, and recommendations.

Lead: Test lead (all contribute)

Output: Final figures, report, and slides

3.6 Time Frame

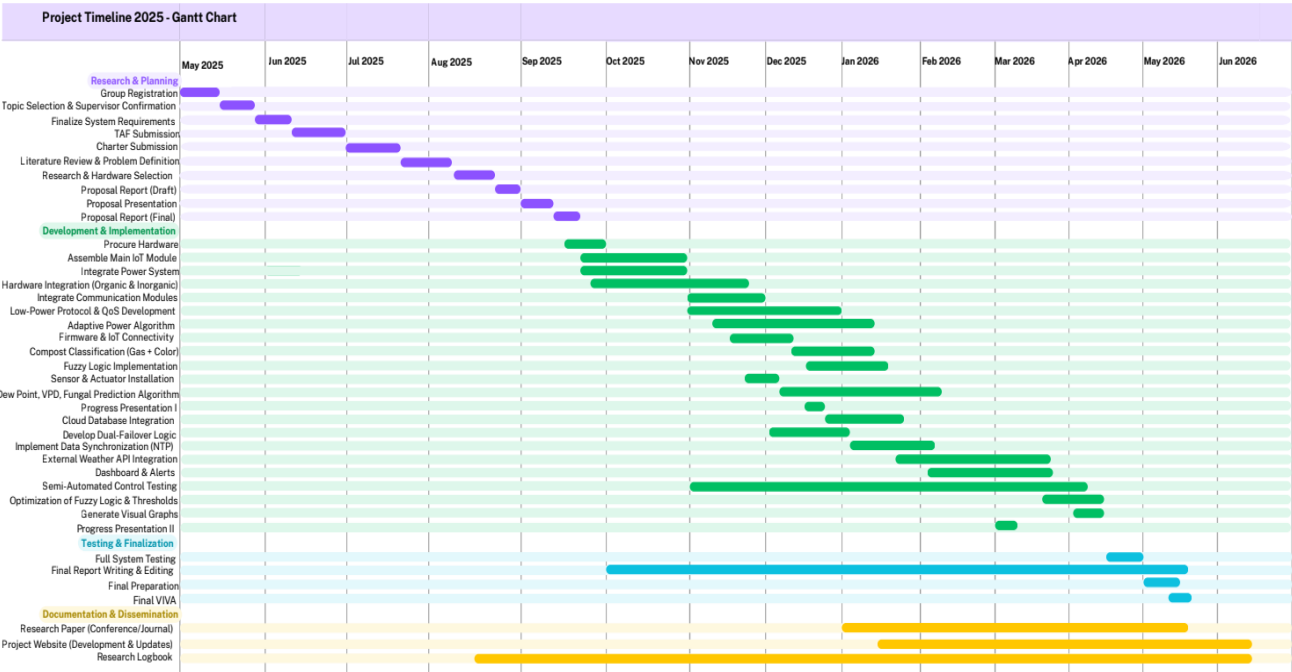


Figure 3.2: Gantt Chart

4. Project requirements

This section provides a detailed breakdown of the requirements for the proposed software and hardware solution. These requirements define the system's core functionalities, quality attributes, and interaction points.

4.1 Functional Parameters (how the system behaves)

What the node measures and sends

On every wake, each tank's node reads pH, EC, temperature, and water level, applies the stored calibration, and packs the values into a small UDP message with device ID, time, and a running sequence. If the reading is outside safe limits, it is marked as critical before anything else is sent.

How energy shapes behavior (VTS-3)

The node watches its battery voltage.

- In High state it samples normally and pushes updates frequently.
- In Medium it slows non-critical updates and batches them.
- In Low it sends alerts only, puts everything else in a buffer, and spends most time in deep sleep.

How messages are treated (QoS)

Critical events (pump off, critically low water, extreme pH/EC) jump the queue, request an ACK, and are retried a few times. Routine readings are allowed to wait or bundle together when power is tight.

What happens when Wi-Fi fails (failover)

Wi-Fi is used first. If several alert ACKs time out, the node enables GSM (SIM800L) just long enough to deliver the alert (data or SMS), then turns GSM off again and returns to Wi-Fi as soon as short health pings succeed.

How time stays aligned

When energy and link quality are good, the node runs SNTP to correct its clock. Every packet is timestamped at the source so the server can line up data from the two tanks.

What happens with no network

If neither Wi-Fi nor GSM can send, the node buffers recent data locally (at least a few hundred records) and uploads it automatically when connectivity returns.

What the server and dashboard do

The server checks the CRC, stores the packet with the arrival time, and updates a web dashboard: live tiles for pH/EC/temperature/level, battery and link status, and a clear alerts panel showing what happened and when.

4.2 Non-Functional Parameters (how well it should work)

Energy use

In ordinary sun/cloud cycles, the node should run 24/7 without brownouts. Over a day, we expect 20–40% lower communication energy and noticeably fewer minutes with the radio on than a simple fixed-rate, TCP-style design. We will report mWh/day and radio-on minutes/day to back this up.

Alert delivery

If something goes wrong in the system, the alert should get through. Even with patchy Wi-Fi, the design aims for at least 99% of critical alerts delivered, with alerts typically reaching the dashboard in ≤ 5 s on Wi-Fi and ≤ 15 s when GSM is used.

Time alignment

Data from the organic and inorganic tanks should line up closely in time. Across a 24-hour window, the difference between tank timestamps should stay within ± 200 ms, so graphs and comparisons are trustworthy.

Scalability and smoothness

One modest server should comfortably handle around 10 nodes (2 tanks each) with quick page refreshes (< 2 s) and low CPU load. Adding another tank should mean adding IDs and thresholds, not rewriting code.

Maintainability

Thresholds (alert limits, VTS-3 voltages, phone numbers) live in config files, not hard-coded. The firmware and backend are modular and commented, with a short README that explains how to deploy and how to change settings safely.

Security and data integrity

Packets carry a CRC so damaged messages are rejected. Payloads are protected with AES-128-GCM using a pre-shared key, and the dashboard is served over HTTPS with password login.

4.3 User Requirements

- Real-time monitoring: View live pH, EC, temperature, water level, plus battery level and network status (Wi-Fi/GSM) on the dashboard.
- Alert notifications: Receive immediate alerts for critical events (e.g., pump failure, low water level, extreme pH/EC) via dashboard banner and optional SMS; allow the user to acknowledge and see alert history.
- Historical trends: Browse daily/weekly/monthly charts per sensor and export CSV for analysis.
- Simple UI: Clear, mobile-friendly layout with color cues (green/yellow/red) and minimal clicks to key views.

4.4 System Requirements

Hardware:

- Microcontroller: An ESP32 development board (e.g., ESP32-WROOM-32).
- Communication Modules: Integrated Wi-Fi module and a separate GSM module (e.g., SIM800L).
- Power System: A solar panel, battery, and charge controller.

Software:

- **Firmware:** Developed in C++ on the Arduino IDE or PlatformIO.
- **Backend:** A cloud-based server (e.g., Firebase, AWS) or a local server to host the database and API.
- **Frontend:** Mobile application for user interaction.
- **Database:** A database to store sensor data (e.g., MongoDB, PostgreSQL).

4.5 Use Cases

Use Case 1 — Monitor system status (User)

The user opens the dashboard and sees live tiles for pH, electrical conductivity (EC), temperature, and water level, plus battery level and whether the device is on Wi-Fi or the mobile network. Charts refresh automatically every few seconds.

Use Case 2 — Receive and acknowledge a critical alert (User/System)

When a reading crosses a safety limit, the device marks it as a critical alert and sends it immediately. The dashboard shows a clear banner (and optionally a text message); the user opens the alert, reads the suggested action, and clicks **Acknowledge**.

Use Case 3 — Network outage and recovery (System)

If Wi-Fi stops working, the device detects the failures, switches to the mobile network, and sends a “Wi-Fi down” notice along with any critical alerts. It keeps testing Wi-Fi and automatically switches back once it is stable, logging “Wi-Fi restored.”

Use Case 4 — Low-power mode (System)

When the battery falls below the low threshold, the device saves energy by batching or temporarily holding routine readings. Critical alerts are still sent right away, and the dashboard shows a “Low-power” badge until the battery recovers.

4.6 Test Cases (Tentative)

Standard data transmission

With normal Wi-Fi, the device sends regular data and status messages. The dashboard tiles and charts update as expected, and the server confirms the messages pass integrity checks (no missing sequence numbers, no checksum errors).

Failover to mobile network (GSM)

Turn off the Wi-Fi router. The device detects the failure, switches to the mobile network, and delivers the alert (as data and/or SMS). When Wi-Fi becomes stable again, the device switches back automatically and logs both the outage and the recovery.

Power-aware prioritization

Force the device into Low-power state. A critical alert (for example, low water level) is sent immediately. Routine readings are grouped or temporarily held to save energy. The device remains running (no brownout).

Energy logging

Run the system for 24 hours on solar or a bench power supply. Record total energy used per day (mWh/day) and the total minutes the radio was active. Verify the results meet the project's energy-saving targets.

Time synchronization

Operate the two tank nodes for 24 hours. Calculate the time difference between matching readings from each tank. Confirm the timestamp difference stays within ± 200 milliseconds.

4.7 Wireframes (Tentative)

Dashboard View: A clean, single-page view with widgets displaying real-time sensor data (e.g., pH, EC, temperature) with color-coded alerts (e.g., green for normal, red for critical). It will also show the current battery percentage and communication status (Wi-Fi or GSM).

Historical Data View: A separate page with interactive charts (e.g., line graphs) that allow the user to view historical sensor data trends over different time periods (day, week, month).

Alerts/Notifications View: A log page that displays a chronological list of all critical events and system alerts received, including timestamps.

5. DESCRIPTION OF PERSONNEL AND FACILITIES

This section summarizes who does what, and the physical/digital resources we will use to execute and validate the system.

5.1 Personnel

- **Lead (Energy-Efficient Networking Module)**

Designs and implements the UDP-based lightweight protocol (ALERT/STATE/DATA), device-side QoS, Voltage-Tiered Scheduler (VTS-3), Wi-Fi, GSM failover, and opportunistic SNTP on ESP32; sets up energy and reliability measurement scripts and evaluation logging.

- **Hardware & Hydroponics Lead**

Builds the solar + Li-Po + charger power path; integrates and calibrates pH/EC/temperature/water-level sensors; manages enclosures, cabling, waterproofing, and safety; maintains dual NFT tanks (organic/inorganic) for field tests.

- **Backend & Dashboard Lead**

Implements Python FastAPI UDP **ingestor (uunicorn)** with packet validation (CRC, sequence); configures PostgreSQL schema and retention; develops React dashboard (live tiles, charts, alerts, history + CSV export); deploys HTTPS reverse proxy and credentials.

- **Test & Documentation Lead**

Authors the test plan, runs bench + field trials (induced Wi-Fi loss, low-battery phases), collects datasets, produces plots/tables; maintains user/admin guides and calibration records.

- **Supervisor**

Provides technical oversight, milestone reviews, and ethical/safety approval for lab and field activities.

5.2 Facilities

- **Electronics & Test Equipment (SLIIT labs)**

ESP32-WROOM-32 boards; SIM800L modules + antennas; 20–30 W solar panels; Li-Po batteries (≥ 2200 mAh) + charger/MPPT; voltage/current sensing (e.g., INA219); multimeters, soldering stations, DC bench supply, breadboards; optional oscilloscope and USB power meter for energy profiling.

- **Hydroponics Setup**

Two NFT tanks (organic & inorganic), nutrient reservoirs, pumps, tubing; pH/EC probes with conditioning boards; DS18B20 temperature probe; float switch for water level; calibration solutions (pH 4/7/10, EC/TDS standards).

- **Networking & Comms**

Dedicated Wi-Fi AP (configurable SSID); active SIM for GSM data/SMS; UPS-backed lab router for controlled outage tests.

- **Compute & Software**

Laptops/PCs with Arduino core / ESP-IDF for firmware; Python 3 + FastAPI (uvicorn) for the UDP ingestor; PostgreSQL database; React for the dashboard; Caddy/Nginx for HTTPS; Git for version control; spreadsheet/plotting tools for analysis.

- **Institutional Support**

Access to SLIIT laboratories (electronics + networking), campus internet/Wi-Fi, and library databases for literature; lab booking and safety PPE as per institute policy.

- **Data & Security**

Secure server instance (on-prem or VM) with firewall rules; AES-128-GCM app-layer encryption for payloads; HTTPS with password authentication; daily database backups.

6. BUDGET AND BUDGET JUSTIFICATION

This section provides a detailed breakdown of the estimated costs for the project in Sri Lankan Rupees (LKR). The budget has been prepared to be as cost-effective as possible while ensuring the use of reliable components and services.

6.1 Estimated Project Budget

Category	Item	Quantity	Unit Cost (LKR)	Total Cost (LKR)
Hardware	ESP32 Microcontroller	1	3,000	3,000
	SIM800L GSM Module	1	2,000	2,000
	Solar Panel (5W)	1	2,000	2,000
	Li-Po Battery (3.7V, 2000mAh)	1	1,000	1,000
	TP4056 Charger Module	1	250	250
	pH Sensor & Module	1	2,000	2,000
	EC Sensor & Module	1	2,500	2,500
	DS18B20 Temp Sensor	1	350	350
	Water Level Sensor	1	250	250
	Assorted Breadboards & Wires	1	1,200	1,200
	Subtotal Hardware			14,550
Software/Services	Cloud Platform (e.g., Firebase)	1	0*	0
	SIM Card & Data Plan	1	3,000	3,000
	Subtotal Software/Services			3,000
Testing & Prototyping	Multimeter	1	3,500	3,500
	Solder, Tools & Consumables	1	1,000	1,000

	Subtotal Testing			4,500
Grand Total				22,050
Contingency (10%)				2,205
Final Budget				24,255

Table 6.1: Estimated Project Budget

6.2 Budget Justification

The project's budget is justified by its focus on leveraging low-cost, readily available hardware and prioritizing open-source software solutions. The primary hardware components, such as the ESP32 and various sensors, are standard in the local market and have been selected for their affordability and robust capabilities.

The most significant and justifiable expense is the inclusion of the GSM module, which is critical for addressing the core research problem of network unreliability. This component directly enables the dual-failover mechanism, a key innovative aspect of the proposal that ensures a high level of system resilience and reliability.

The inclusion of a 10% contingency fund is a crucial measure for managing project risks. This reserve accounts for potential unforeseen costs, such as the need for replacement components, unexpected subscription fees, or additional materials required during the prototyping and testing phases. This proactive financial planning demonstrates a sound and realistic approach to project management.

7. CONCLUSIONS AND RECOMMENDATIONS

7.1 Conclusions

This project presents a low-power, resilient communication and power-management module for off-grid, solar-powered hydroponics. It directly targets the pain points highlighted in the background and literature: transport overhead with TCP, fixed-rate duty cycling, single-link fragility, and unsynchronized data streams. The design centers on a UDP-based lightweight application-layer protocol with selective ACK/NAK for alerts, a Voltage-Tiered Scheduler (VTS-3) for energy-aware operation, device-side QoS for priority handling, and dual-link failover (Wi-Fi primary, GSM backup). Time alignment is maintained with opportunistic SNTP and source timestamps so the two tanks (organic vs. inorganic) remain comparable.

Expected outcomes, to be validated in the evaluation:

- Energy: 20–40% reduction in communication energy and $\geq 30\%$ fewer radio-on minutes/day versus a conventional TCP + fixed-rate approach.
- Reliability: $\geq 99\%$ delivery of critical alerts during connectivity disturbances; typical alert latency ≤ 5 s on Wi-Fi and ≤ 15 s on GSM.
- Scientific validity: inter-tank timestamp skew $\leq \pm 200$ ms over 24 hours, enabling fair comparisons.

Overall, the system is practical, low-cost, and extensible: it improves reliability, data integrity, and sustainability for dual hydroponic tanks and can be adapted to broader smart-agriculture scenarios.

7.2 Recommendations

1. **Broaden deployments and stress-test at scale**

Pilot across multiple tanks and sites (different Wi-Fi quality, shading, nutrient regimes) to test robustness, tune VTS-3 thresholds, and refine failover timings.

2. **Lightweight prediction for energy-aware control**

Replace static VTS-3 thresholds with a lightweight predictor (e.g., day-ahead solar heuristic or simple linear model) to anticipate low-energy periods and pre-batch routine data even earlier.

3. **Long-range option for sparse farms**

For very large sites, add a LoRaWAN uplink for routine `DATA` while keeping Wi-Fi/GSM for `ALERT`. This hybrid retains low energy for background telemetry without sacrificing alert latency.

4. **Harden security and operations**

Keep AES-128-GCM for payloads and HTTPS on the dashboard; add key **rotation**, role-based access, and nightly database backups. Provide a simple **config panel** for thresholds and phone numbers to reduce configuration errors.

5. **Maintenance & calibration playbook**

Document sensor calibration intervals, battery health checks, and SIM data/SMS quotas. Provide a one-page field checklist (spares, tools, recovery steps).

6. **Path to a farmer-ready kit**

Package the design as a low-cost kit (ESP32 + SIM800L + sensors + solar + enclosure) with a guided setup and hosted dashboard tier, enabling small/medium farms to adopt quickly.

7. **Cross-domain reuse**

Apply the same transport + scheduling + failover pattern to aquaculture, remote environmental stations, and rural utilities where power is scarce and links are intermittent.

REFERENCES

- [1] A. Kaur and T. N. Swamy, *Energy-Efficient IoT Systems: Integrating Low-Power Protocols and Adaptive Algorithms for a Greener Future*. ResearchGate, 2024.
- [2] D. van Leemput, J. Famaey, J. Hoebeke, and E. de Poorter, “Energy-aware adaptive scheduling for battery-less 6TiSCH routers in industrial wireless sensor networks,” *IEEE Access*, vol. 12, pp. 180034–180047, 2024.
- [3] M. S. W. M. M. I. E. E. S., “A Dynamic Framework for Internet-Based Network Time Protocol,” *Sensors*, vol. 24, no. 2, p. 691, 2024.
- [4] K. Charoenchaiprakrit, W. Piyarat, and K. Woradit, “Optimal data transfer of SEH-WSN node via MDP based on duty cycle and battery energy,” *IEEE Access*, vol. 9, pp. 82947–82965, 2021.
- [5] N. G. Adetunji, T. E. Okoro, F. E. Ibeh, and O. T. Oluwajoba, “Performance Evaluation of UDP-Based Data Transmission with Acknowledgment for Various Network Topologies in IoT Environments,” *Electronics*, vol. 13, no. 18, p. 3697, 2024.
- [6] C. A. Boano, “Communication Failover Strategies for Dependable Smart Grid Operation,” in *Proc. 2nd Int. Conf. Wireless Mobile Comput., Netw. Commun. (WiMob)*, 2021.
- [7] “Emerging indoor photovoltaics for self-powered and self-aware IoT towards sustainable energy management,” *Semantic Scholar*, 2023.
- [8] “Task Scheduling for Simultaneous IoT Sensing and Energy Harvesting: A Survey and Critical Analysis,” *Semantic Scholar*, 2022.
- [9] M. E. Yuksel, “Power consumption analysis of a Wi-Fi-based IoT device,” *Istanbul Univ. J. Electr. Electron. Eng.*, 2020.
- [10] R. A. F., A. W. C., L. L. S., I. R. G., L. I. G., I. C., and B. A. S., “Effects of IoT Communication Protocols for Precision Agriculture in Outdoor Environments,” *ResearchGate*, 2024.
- [11] T. Melkysedek, E. Hesti, and I. Salamah, “Design and build hydroponic installations and applications using IoT-based multisensors with solar panel electrical energy,” *Indonesian J. Electron. Instrum. Syst. (IJEIS)*, 2023.
- [12] A. T. H., Z. Z., F. M., K. C., and B., “An Improved Network Time Protocol for Industrial Internet of Things,” *Sensors*, vol. 22, no. 13, p. 5021, 2022.

- [13] S. Garg, “Redundancy as a Measure of Fault-Tolerance for the Internet of Things: A Review,” *J. Wireless Netw. Commun.*, vol. 3, no. 2, pp. 102–110, 2021.
- [14] A. K. Rai and A. K. Sarma, “Quality-of-Service Prediction Techniques for Wireless Sensor Networks,” *J. Phys.: Conf. Ser.*, vol. 1950, no. 1, p. 012082, 2021.
- [15] S. A. Gholami, S. A. Gholami, and H. K. Kim, “Solar panel monitoring and energy prediction for smart solar system,” *ResearchGate*, 2020.
- [16] H. S. Sishodia, M. L. Singh, and S. Kumar, “IoT-based Smart Hydroponics System,” in *Proc. Int. Conf. Adv. Comput. Commun. Eng.*, 2021, pp. 1–5.
- [17] R. S. H., K. S., E. D., and R. K., “Overview of Time Synchronization for IoT Deployments: Clock Discipline Algorithms and Protocols,” *Sensors*, vol. 20, no. 20, p. 5928, 2020.
- [18] R. K. Singh, M. Aernouts, M. De Meyer, M. Weyn, and R. Berkvens, “Leveraging LoRaWAN technology for precision agriculture in greenhouses,” *Sensors*, vol. 20, no. 7, p. 1827, 2020.
- [19] N. Hasan, M. Ali, A. Barradas, and N. Correia, “Cross-layer optimization for reliability improvement of data delivery in 6LoWPAN-based networks,” in *Proc. 14th Annu. Mediterranean Ad Hoc Networking Workshop (MedHoc-Net)*, Vilamoura, Portugal, 2015, pp. 1–7, doi: 10.1109/MedHocNet.2015.7173162.
- [20] N. Muleta and A. Q. H. Badar, “Study of energy management system and IoT integration in smart grid,” in *Proc. 1st Int. Conf. Power Electron. and Energy (ICPEE)*, Bhubaneswar, India, 2021, pp. 1–5, doi: 10.1109/ICPEE50452.2021.9358769.
- [21] C. Changqing, L. Hui, and H. Wenjun, “Internet of agriculture-based low-cost smart greenhouse remote monitor system,” in *Proc. Chinese Automation Congress (CAC)*, Xi’an, China, 2018, pp. 3940–3945, doi: 10.1109/CAC.2018.8623230.
- [22] G. Balabanov and S. Kolev, “IoT networks QoS guarantee,” in *Proc. 58th Int. Sci. Conf. Information, Communication and Energy Systems and Technologies (ICEST)*, Niš, Serbia, 2023, pp. 109–112, doi: 10.1109/ICEST58410.2023.10187341.